```
  let(:ticket) { BerlinTransitTicket.new }

  before do
    # These values depend on `let` definitions
    # defined in the nested contexts below!
    #
    ticket.starting_station = starting_station
    ticket.ending_station   = ending_station
  end

  let(:fare) { ticket.fare }

  context 'when starting in zone A' do
    let(:starting_station) { 'Bundestag' }

    context 'and ending in zone B' do
      let(:ending_station) { 'Leopoldplatz' }

      it 'costs €2.70' do
        expect(fare).to eq 2.7
      end
    end

    context 'and ending in zone C' do
      let(:ending_station) { 'Birkenwerder' }

      it 'costs €3.30' do
        expect(fare).to eq 3.3
      end
    end
  end
end
```

Let's trace through what happens when RSpec runs the first example:

1. The before hook on line 4 begins to run.
2. We refer to ticket on line 8, which jumps us to the let definition on line 2 to create that object.
3. We reference starting_station on line 8, which jumps to the let definition on line 15 and back.
4. On line 9, we reference ending_station, which jumps to the inner let definition on line 18 and back.
5. The before hook completes, so now we start running the example on line 20.
6. The example references fare, which jumps to the let definition on line 12 and back.
7. The expectation runs, and the example completes.